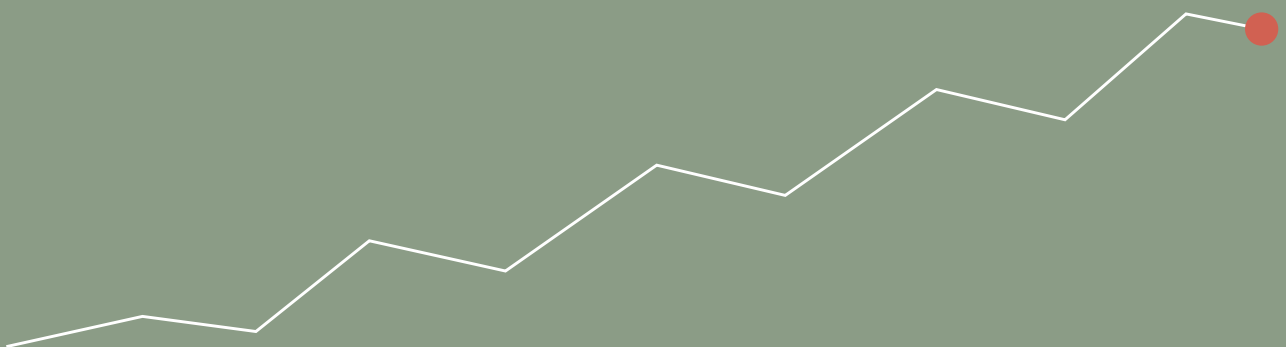


Keepr *Analytics*

The operations console of a private family messenger: see usage, cost and sensitive access — without ever touching conversation content, or appearing in the app.



5

signal sources,
each with its own
collection mode

2

server-side locks —
database rules
and functions

0

message content
collected by
telemetry

1

designer-developer,
from specification
to deployment

Operate a private service *without looking inside.*

Keepr is a messaging PWA used by a tight circle — family and close friends. Once a service is in production, you need to know whether it is used, what it costs, and who touches administration. The hard part: getting those answers without building a tool that watches people.

Personal data	Conversations are private. Observability must run on counters and metadata, never on content.
Near-zero cost	A personal project on usage-based cloud billing: every write counts, and no pipeline should run “just in case”.
A single operator	No on-call, no team. Panels must be readable at a glance and reliable without daily maintenance.
A real financial risk	A budget cut-off protects the project. Observability must never be able to stop that cut-off from working.
Zero product impact	No screen, no latency, no analytics bug should be perceptible to people using the app.

ANSWER

Is it used?

Which parts of the app are alive, which are neglected, day after day.

ANSWER

What does it cost?

Where consumption sits against budget, and which services pull it.

ANSWER

Who administers?

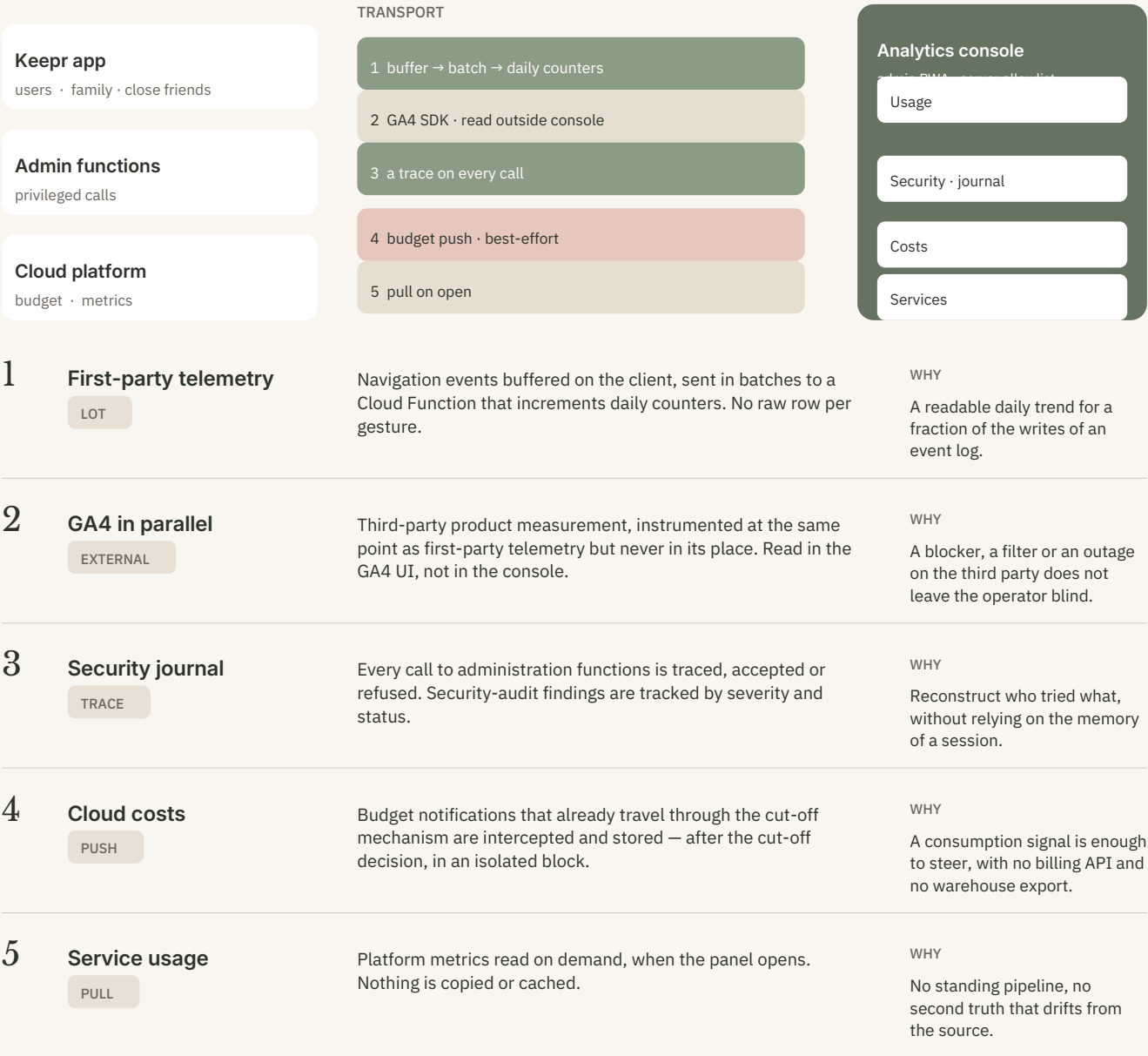
Every attempt to reach admin functions, accepted or refused.

WHAT THE CONSOLE IS NOT

- A hidden screen inside the family app
- A conversation reader
- A parallel data warehouse
- A link in the cut-off mechanism

Five signals, *five collection modes.*

Each source is handled with the mode that fits it — batch, push, pull, external. None is a bridge to another: if one falls, the others stay readable.



Every choice *has a price, owned.*

Trade-offs are written as architecture decisions: the option kept, the one set aside, and what we accept to lose in return.

DECISION	ALTERNATIVE SET ASIDE	REASON	COST ACCEPTED
Two independent usage measures	Everything in GA4, or everything first-party	Resilience: either channel can fail without blinding the other.	Double instrumentation; figures not reconcilable with each other.
Aggregated daily counters	Raw event log	Minimal writes and storage; data minimisation by construction.	No individual-journey analysis — on purpose.
Batch send, no waiting	One write per interaction	Cost under control; a tracking failure never blocks the UI.	A few events lost if the app closes before the flush.
Costs via budget notifications	Billing API, warehouse export	Reuses an existing flow; zero extra infrastructure.	Granularity and freshness limited to what the alerts publish.
Cost writes off the critical path	Tracing baked into cut-off logic	Financial protection outranks visibility.	A reading may be missing; the cut-off, never.
Service metrics on pull	Scheduled collection and storage	Always-fresh data, no standing job.	One call per open; history limited to source retention.
Console = a distinct PWA	A hidden screen in the app	No admin surface exposed; the app bundle stays lean.	Two installable apps to host side by side.
Access checked server-side, twice	Hiding the screen on the client	A modified client is not enough to read or write.	An allowlist to keep consistent in two places.

GUIDING PRINCIPLE

Observability must never cost more, or be more fragile, than the service it observes.

Trust never comes *from the browser.*

Access to the console is not a hidden screen: it is a server check, applied twice, and traced. The frontend displays; it decides nothing.

CLIENT

Console UI

Can be altered by anyone.
Treated as untrusted by default.

LOCK 1

Database rules

Analytics collections refuse any
read or write outside the
allowlist.

LOCK 2

Cloud Functions

Every admin function re-checks
identity before acting.

TRACE

Journal

Accepted or refused, every call
leaves a readable trace.

MINIMISATION — WHAT TELEMETRY CONTAINS

COLLECTED

Name of the app space visited

Aggregate counter per day

Budget amounts and thresholds

Service health metrics

Timestamp and outcome of admin calls

NEVER COLLECTED

Message text

Attachments, media, transcripts

Per-user individual journeys

Copies of conversations, even partial

Secrets or tokens in logs

Fire-and-forget

A telemetry send failure is absorbed silently
and locally. The user never sees it.

European hosting

Stack hosted in a European region. No
secondary warehouse off the platform.

Minimal public surface

Identifiers, service accounts and operational
thresholds appear in no public document.

Ship alone, *with team-grade guardrails.*

Code is produced with an AI agent (Claude Code), steered like a team: scoped briefs, contradicted reports, evidence required. Rigour comes from the process, not from trust.

01

Reconnaissance

Read the source before any write. No fix based on a guess.

02

Brief

Scope, forbidden files and acceptance criteria written before execution.

03

Staging

Automatic deploy, e2e tests in CI, check of the build actually served.

04 · GATE

Validation

Production only after a manual check on a device.

Rules before interface	Access rules are deployed before the frontend that depends on them — never the other way around.
Named sensitive files	Every diff is checked against an explicit list of untouchable files: rules, hosting config, CI pipelines, cut-off logic.
Bundle isolation	The console imports no shared component that would bloat its bundle; pieces are reimplemented locally.
One agent session at a time	The repo is shared with the main product: no concurrent runs.
Incidents written down	Known traps and incidents are documented in the repo, with cause and the rule that follows.

- STACK

 - Firebase Hosting, Cloud Functions (Node 22), Firestore
 - Pub/Sub, Cloud Monitoring, Secret Manager
 - React, Vite · installable PWA
 - GitHub Actions · Playwright e2e
- OPEN WORK

 - CI run tracking inside the console
 - Per Cloud Function detail over 24 h and 7 days
 - Unit tests for panel error states
 - Branch protection on production

DESIGN & DELIVERY

Mathieu El Almawie

Business Analyst MOA · PO · product design and architecture of Keepr
Site mthlabs.github.io/Portofolio Contact mathieu.elalmawie@gmail.com